Spring 2026 - Semester Progress Report
Recursive Language Models and Semantic Caching

Engin Deniz Dogu
edogu@stevens.edu

05.15.2026

Course: CS 800 - Special Problems in CS
Collaborator: Adarsh Vatsa
Supervisor: Prof. William Eiers

1 Introduction

Recursive Language Models (RLMs) are a promising way to apply LLMs to long-context tasks by decomposing large inputs into smaller model calls. However, this recursive structure creates repeated subquestions, repeated document chunks, and repeated intermediate summaries. Each repeated call adds cost, latency, and another opportunity for an unsupported answer.

Existing approaches only partially address this issue. Standard LLM calls do not reuse previous work; string caches miss paraphrases; and pure vector-similarity caches can return unsafe matches when similar-looking queries ask for different information. Retrieval systems reduce context length, but they do not by themselves prevent repeated synthesis or ensure that a reused answer came from the correct source.

This project proposes a scoped semantic cache for RLM-style workflows. The system stores previous answers, retrieves candidate matches with local embeddings, verifies semantic equivalence before reuse, and gates cache hits by source and document-set identity. It also records cost, provenance, benchmark metadata, and extracted knowledge so reuse can be audited.

Current results show that this approach can reduce repeated API usage and estimated cost while preserving measured accuracy on repeated workloads. A warm-cache RULER v2 run eliminated API calls for the repeated workload while matching cold-cache accuracy, and LegalBench/CUAD warm-cache runs substantially reduced cost, tokens, and runtime while preserving cache-enabled answer accuracy. The project is still incomplete, and broader evaluation on the modified LongBench-v2 setup remains future work.

The semester work progressed from understanding the RLM codebase to building and evaluating a two-stage semantic cache. By the current point in the project, the repository includes RLM execution notes, early local/API experiments, a scoped cache implementation, benchmark runners for RULER v2, NoLiMa, synthetic cache-mode tests, LegalBench/CUAD, and a prepared modified LongBench-v2 cache dataset.

2 Related Work

Several recent papers study the same broad problem as this project: how to make LLMs reliable and useful when the relevant input is much longer than a single convenient prompt. Recursive Language Models (RLMs) propose an inference strategy where a model treats the prompt as part of an external environment and recursively decomposes long inputs into smaller model calls [1]. MemGPT approaches the same context-window limitation through operating-system-inspired memory management, moving information between active context and external storage [2]. Long-context benchmarks show why these systems are needed: LongBench introduced a bilingual multitask benchmark for long-context understanding [3]; RULER showed that simple needle-in-a-haystack retrieval is too shallow and that models degrade as context length and task complexity increase [4]; NoLiMa further showed that long-context retrieval becomes much harder when questions and evidence do not share obvious lexical overlap [5]; and LongBench-v2 moves toward more realistic long-context reasoning with multiple-choice questions over contexts ranging from thousands to millions of words [6]. This project is closest to RLMs because it studies

repeated recursive model calls, but it focuses on an infrastructure layer that those systems need: safe reuse of prior subcall results. Unlike the benchmark papers, this work does not only evaluate long-context failure modes; it builds a cache mechanism intended to reduce the cost and runtime of systems that operate in those settings.

Other papers use related caching and memory ideas, but often for different deployment problems. GPTCache introduced an open-source semantic cache for LLM applications, reusing previous answers for semantically similar prompts to reduce latency and cost [7]. MeanCache studies semantic caching for LLM web services with attention to user-centric behavior and privacy [8]. VectorQ improves semantic prompt caching by adapting similarity thresholds instead of relying on one fixed cutoff [9]. RAGCache targets retrieval-augmented generation systems by caching intermediate knowledge states to improve serving latency and throughput [10]. Krites is especially relevant because it uses an LLM judge to verify borderline semantic-cache candidates before promoting them for future reuse [11]. This project also uses semantic reuse and an evaluator-style verification step, but the target problem is different: it applies caching to RLM-style and benchmarked autonomous workflows, where source-context isolation, cache-route diagnostics, persistent benchmark state, and answer provenance are central requirements rather than optional service-level features.

3 Method

The proposed method is a scoped semantic cache for repeated LLM calls. Each cache entry stores the original query, generated answer, query embedding, source context, model metadata, provenance metadata, and optional extracted facts. More formally, a cache entry can be written as:

$$e_i = (q_i, a_i, z_i, h_i, s_i, m_i, p_i)$$

where q_i is the stored query, a_i is the answer, $z_i = \text{embed}(q_i)$ is the embedding of the query, h_i is the hash of the exact source chunk, s_i is the hash of the active document set, m_i records model/cost metadata, and p_i records provenance or verification metadata. For direct RLM-style calls over a given context c, the system computes $h = H(\text{normalize}(c))$ and only compares entries in the same source bucket. For retrieval-style calls over an ingested document collection D, the system computes a data-scope hash:

$$s = H(\text{sorted filenames}, \text{normalized file contents}, \text{chunk size}, \text{overlap})$$

This hash is used as an eligibility gate. A cached answer is not allowed to serve a retrieval query unless it was produced from the same active document set. This design is stricter than a global query cache, but it is necessary because two benchmark samples can contain the same question text while requiring different answers from different document contexts.

The cache lookup proceeds in stages. First, the system checks for exact matches inside the eligible source or data scope:

$$\text{hit}_{\text{exact}}(q, e_i) = \mathbb{1}[\text{lower}(q) = \text{lower}(q_i)] \wedge \mathbb{1}[\text{scope}(q) = \text{scope}(e_i)]\,.$$

If no exact match exists, the system performs a semantic search over cached query embeddings. Given a new query q, it computes $z = \text{embed}(q)$ and scores each eligible candidate with cosine similarity:

$$\text{sim}(q, q_i) = \frac{\text{dot}(z, z_i)}{\text{norm}(z)\,\text{norm}(z_i)}.$$

FAISS is used to retrieve the top candidates efficiently. A candidate enters the semantic verification stage only if its similarity is above a configured threshold. The second stage is an evaluator-model check, called the "Sniper" in the project notes. Instead of trusting vector similarity alone, the evaluator decides whether the new query and cached query ask for the same information. This two-stage design was chosen because exact caching is too conservative, but vector-only semantic caching is too risky: questions such as "include timeout errors" and "exclude timeout errors" can be close in embedding space while requiring opposite answers.

The system also maintains a knowledge cache. When an answer is generated, the cache extracts structured facts that can be represented as scoped triples:

$$f_j = (\text{subject}_j, \text{relation}_j, \text{object}_j, s_j, h_j).$$

These facts are embedded and indexed separately. A later query may match a fact even when it is not equivalent to the original cached query. The intended use is cross-query reuse: a broad setup question can populate reusable facts, and later narrower questions can retrieve those facts. The current implementation records these facts and indexes them, but the benchmark results show that knowledge-hit behavior still needs stronger span support, verifier status, and route diagnostics before it can be treated as a final contribution.

On a cache miss, the system runs a retrieval and synthesis pipeline. The active corpus is chunked and embedded. FAISS retrieves a broad candidate set, a local reranker filters and ranks the candidates, and the executor model synthesizes an answer from the selected evidence. The generated answer is then checked for grounding, optionally verified through a second model, stored in the cache, and used to extract new facts. The high-level miss path is:

$$\begin{aligned} &\text{query} \rightarrow \text{FAISS retrieval} \rightarrow \text{reranking} \rightarrow \text{LLM synthesis} \\ &\quad \rightarrow \text{grounding/consensus} \rightarrow \text{cache write} \rightarrow \text{fact extraction} \end{aligned}$$

There are several reasonable alternatives to this design. One alternative is a simple string cache; this is safe but misses paraphrases and therefore gives low hit rates in realistic recursive workflows. Another alternative is a pure embedding cache; this improves hit rate but can return false positives when two queries are lexically similar but semantically different. A third alternative is to use only retrieval-augmented generation without answer caching; this avoids cache collisions but still pays for repeated synthesis. The chosen design combines exact matching, vector retrieval, evaluator verification, source scoping, and persistent benchmark state because the project goal is not only to reduce cost, but to reduce cost without allowing cached answers to cross into the wrong document or benchmark sample.

4 Project Development

4.1 Initial RLM Exploration and Early Experiments

At the start of the semester, the work focused on understanding the RLM codebase and reproducing simple examples. This included setting up VPN and JARVIS/HPC access, reading the repository structure, and constructing an execution-flow diagram to make the system easier to reason about. Figure 1 in the Appendix summarizes the root RLM process, how prompts are loaded into a local environment, how sub-RLM calls are spawned, and how final answers are returned.

The initial architecture notes captured several important details:

- An RLM instance has parameters such as backend, environment, depth, `max_depth`, `max_iterations`, system prompt, persistence, and logging.
- The root RLM runs with `depth=0`; sub-RLMs run at deeper levels.
- The local REPL environment allows the model to manipulate the prompt as a variable, split context, call sub-RLMs, and combine intermediate outputs.
- The default behavior falls back to a regular LM completion when maximum depth is reached.
- Persistence changes whether the environment is reused across calls.
- The current RLM implementation appeared to support only `max_depth=1` in practice, even when higher values were configured.

This early analysis raised questions about the practical difference between increasing `max_depth` and increasing `max_iterations`, whether deeper recursive calls should run sequentially or in parallel, and whether a caching layer could make repeated subcalls reusable without returning answers from the wrong source context. Initial experiments also tested an Ollama client, local Qwen/Ollama execution, and API-based RLM workflows. The recorded quickstart runs included a GPT-style API run with a 400K input / 128K output budget at about 120 seconds, and a local `qwen2.5:7b` Q4 Ollama run with a 128K input / 8K output budget at about 43 seconds.

Small synthetic RLM-style tasks were then used to check reliability on repeated long-context search. These included finding hidden numbers and answering context-grounded questions where the supplied context intentionally conflicted with real-world knowledge. The experiments showed that simple extraction could work, but reliability varied by prompt, model, context length, and grounding. This motivated later requirements: benchmark tasks need clear scoring, answers must be tied to the supplied source context, and cache reuse must not cross source boundaries.

4.2 Benchmark Survey and Data

A major part of the semester was surveying long-context and RLM-relevant benchmarks. Candidate benchmarks were collected in `report/Benchmarks.csv` and later organized into implemented and candidate benchmark tracks in `docs/benchmarks.md`.

The benchmark survey included:

- RULER and Needle-in-a-Haystack style retrieval benchmarks.
- NoLiMa, which reduces literal overlap between query and needle.
- LongBench and LongBench-v2 for broader long-context QA/reasoning.
- OOLONG and CorpusQA for aggregation-heavy or corpus-level reasoning.
- Legal and contract benchmarks such as CUAD, ContractNLI, and LegalBench-RAG.
- Memory-oriented benchmarks such as LongMemEval and MemoryAgentBench.

This research changed the evaluation plan. RULER and NoLiMa are useful engineering stress tests, but they are mostly retrieval-focused. LongBench-v2 became the next primary candidate because it offers broader long-context reasoning and deterministic multiple-choice scoring. LegalBench/CUAD became useful for real-domain cache-route debugging because it contains real contracts, clause labels, and answer spans.

The evaluated data so far comes mainly from RULER v2 and LegalBench/CUAD. The prepared RULER v2 data contains 102 samples across `mk_niah_basic`, `mv_niah_basic`, and `qa_basic` at 8,192 and 32,768 token contexts; for example, `qa_basic` asks the system to find the most relevant document index for a target text. The LegalBench/CUAD suite contains 400 route-labeled contract cases across exact, semantic, knowledge, and miss categories, with 60 cases selected in the reported runs. A typical CUAD-style query asks the model to highlight contract language related to a clause such as `Parties`, with the expected answer being a contract span such as `Distributor`.

Two smaller tracks were used mainly for implementation checks. The synthetic cache-mode suite has 23 cases over three small corpora and directly tests exact, semantic, knowledge, and miss behavior; for example, it asks both `What was ACME's ARR in Q1 2023?` and the paraphrase `How much annual recurring revenue did ACME report for Q1 2023?`. NoLiMa is currently represented by a small fixture with two needle templates and one haystack file, so it should be treated as a runner/scorer smoke test rather than a headline benchmark result.

The modified LongBench-v2 setup is the next broader reasoning benchmark being prepared. The local export contains 503 original multiple-choice questions and 503 exact duplicate rows across 462 unique contexts. A pilot cache-suite file contains 1,032 rows: original, exact, semantic rewrite, setup, and knowledge rows. The semantic and setup rows were generated with the Codex SDK using `gpt-5.5` at high reasoning effort, then saved as reviewable CSV/audit files. This design preserves LongBench-v2's deterministic answer labels while adding cache-route structure for exact reuse, paraphrase reuse, setup-driven fact extraction, and later knowledge reuse.

4.3 System Architecture

The central architecture direction became a two-stage semantic cache for autonomous agents and RLM-style workflows. The system is designed around the idea that recursive LLM systems repeatedly ask similar questions over the same or related source material. A simple string cache only catches exact duplicates, while a pure vector cache can produce dangerous false hits. The project therefore uses a two-stage "Dragnet and Sniper" design:

1. The Dragnet stage uses local embeddings and FAISS search to cheaply collect likely similar cached entries.
2. The Sniper stage uses a smaller/evaluator LLM to decide whether the new query is truly semantically equivalent to a cached query.

The implemented architecture in `semantic_cache_system.py` includes:

- `EmbeddingEngine`: local Qwen3 embedding model for query and document embeddings.
- `FAISSIndex`: vector index wrapper with metadata and persistence.
- `Reranker`: local Qwen3 cross-encoder reranker for retrieved documents.

- `ExecutionMetrics`: counters for cache hits, misses, API calls, token usage, cost, and provenance.
- `SemanticCacheController`: exact, semantic, and knowledge cache lookup; document retrieval; synthesis; grounding; consensus; fact extraction; persistence; and data-scope validation.
- `CachePreWarmer`: programmatic cache saturation over templates and chunks.
- `Router`: model routing for simple extraction versus more complex synthesis tasks.
- `AutonomousAgent`: framework-agnostic cached-query facade for agent/RLM-style workflows.

The system uses local Qwen3 models for embedding/reranking and Anthropic Claude models for synthesis/evaluation roles.

4.4 Cache Correctness and Source Isolation

A recurring challenge was that cache reuse can improve efficiency while also introducing correctness risks. The project therefore added multiple isolation mechanisms.

The first layer is `source_chunk_hash`, which identifies the exact source chunk or context that produced a cached answer. This protects direct `cached_query(query, context)` calls by preventing a query over one document from reusing an answer generated from another document.

The second layer is `data_scope_hash`, which identifies the active ingested document set for retrieval-based `search(query)` calls. This was added after discovering a RULER `qa_basic` failure mode: two samples could contain identical question text but different document contexts and different correct document IDs. Query-only exact matching was not safe. The fix made exact, semantic, and knowledge cache hits eligible only when the cached entry's `data_scope_hash` matches the currently ingested document set.

This distinction became an important implementation and reportable research contribution:

- `source_chunk_hash` answers: which exact source context produced this cached answer?
- `data_scope_hash` answers: which active document set was being searched when this answer was produced?

Regression tests were added for scoped exact hits, scoped knowledge hits, scoped persistence, and legacy unscoped cache entries.

4.5 Evaluation Infrastructure

Several benchmark tracks were implemented or prepared so that the cache system could be evaluated reproducibly rather than only demonstrated manually. The repository now includes runners and artifact pipelines for RULER v2, an initial RLM baseline, NoLiMa, a synthetic cache-mode suite, and LegalBench/CUAD; these write predictions, bridge rows, manifests, evaluation reports, and reusable cache state.

The benchmark work also produced a modified LongBench-v2 preparation workflow. It preserves the original multiple-choice labels while adding route-oriented rows for exact reuse, semantic paraphrases, setup questions, and knowledge reuse, so the next stage can evaluate broader long-context reasoning while still measuring cache behavior.

5 Results

The project is still in progress, so the results below should be interpreted as current evidence rather than final claims. The strongest completed evaluations so far measure whether persistent semantic-cache reuse reduces API calls, token usage, estimated cost, and runtime while preserving the measured accuracy of the original cache-producing run.

5.1 Quantitative Results

The strongest RULER v2 comparison currently visible is the cold-cache run on April 23, 2026 and the warm-cache run on April 27, 2026. Both runs use the same 102 selected samples across `mk_niah_basic`, `mv_niah_basic`, and `qa_basic` at 8,192 and 32,768 token contexts.

Run	Mode	Samples	API Calls	Tokens	Estimated Cost	Elapsed Time	Cache Hits	Accuracy
1	Cache, cold start	102	306	513,515	$1.212457	8,017.457 sec	0 / 102	0.8113
2	Cache, warm start	102	0	0	$0.000000	3,764.086 sec	102 / 102	0.8113

The warm RULER run preserved the measured accuracy of the cold run while eliminating API calls and estimated API cost for the repeated workload. Runtime decreased by about 53.1%, although it did not go to zero because the benchmark still performs local orchestration, loading, scoring, and artifact writing.

The by-task results were:

Task and Context	Samples	Accuracy
`mk_niah_basic`, 32,768	17	0.6471
`mk_niah_basic`, 8,192	17	0.9412
`mv_niah_basic`, 32,768	17	0.5882
`mv_niah_basic`, 8,192	17	0.7500
`qa_basic`, 32,768	17	0.9412
`qa_basic`, 8,192	17	1.0000

This result supports the claim that persistent cache reuse can eliminate repeated API cost when the same benchmark workload is rerun, but it also shows that accuracy depends on the underlying retrieval/synthesis quality from the cold run.

The LegalBench/CUAD runs provide a clearer cost comparison across baseline, cold cache, and warm cache modes on 60 cases.

Run	Mode	Cases	API Calls	Tokens	Estimated Cost	Elapsed Time	Answer Accuracy	Cache Hit Rate
1	Baseline	60	360	611,410	$1.962066	4,848.257 sec	0.5833	N/A
2	Cache, cold start	60	285	406,647	$1.278803	3,402.981 sec	0.6667	0.75
3	Cache, warm start	60	30	8,008	$0.009688	659.463 sec	0.6667	1.00

Compared with the baseline, the warm-cache LegalBench run reduced estimated cost by approximately 99.5%, API calls by approximately 91.7%, and runtime by approximately 86.4%, while preserving the cache-mode answer accuracy measured in the cold-cache run.

The synthetic cache-mode suite provides route-level diagnostics rather than a headline benchmark score. The latest recorded run selected 23 cases. It achieved 100% answer accuracy and a 95.65% cache hit rate, but only 43.48% cache-route match rate. This means the system often returned the right answer from cache, but through a different route than the case was designed to test.

Benchmark	Cases/Samples	Answer Accuracy	Cache Hit Rate	Route Match Rate	Estimated Cost
Synthetic cache-mode suite `20260428T153235Z`	23	1.0000	0.9565	0.4348	$0.005834

Benchmark	Cases/Samples	Answer Accuracy	Cache Hit Rate	Route Match Rate	Estimated Cost
LegalBench/CUAD cold cache 20260504T033206Z	60	0.6667	0.7500	0.7500	$1.278803
LegalBench/CUAD warm cache 20260504T042946Z	60	0.6667	1.0000	0.5000	$0.009688

The RULER RLM baseline run `benchmark_artifacts/official_ruler_v2_rlm/20260504T150419Z` evaluated 3 samples, one each from `mk_niah_basic`, `mv_niah_basic`, and `qa_basic` at 32,768 tokens. It reached 1.0 accuracy on those 3 samples with 702,406 total tokens, $0.75049 estimated cost, and 733.477 seconds elapsed. This is a small baseline, so it should be presented as an initial cost-profile comparison rather than a final RLM-vs-cache conclusion.

5.2 Qualitative Results

A successful semantic-cache example appears in the synthetic cache-mode suite. The seed query asks: `What was ACME's ARR in Q1 2023?` The evaluation query asks the same question with different wording: `How much annual recurring revenue did ACME report for Q1 2023?` The expected answer is `$12.4M`. The system correctly served the result as a semantic cache hit, with the answer: `ACME's ARR in Q1 2023 was $12.4M.` This example shows the intended behavior of the two-stage design: exact string matching would miss the paraphrase, but semantic lookup plus verification can reuse the earlier answer.

A failure mode also appears in the synthetic cache-mode suite. In the case `ops_negative_timeout_inversion`, the test expected a miss because the query involved an exclusion/inclusion distinction in the incident policy. The system still served the answer from cache through an exact route because a previous warmed entry already matched the final query text. The returned answer was factually correct: `sandbox timeout warnings should be excluded from the incident review.` However, the route behavior failed the diagnostic test because the case was meant to verify that the system would avoid unsafe semantic reuse. This illustrates an important distinction for future evaluation: answer correctness, cache hit rate, and intended cache route are different metrics and must be reported separately.

The LegalBench/CUAD results show a similar qualitative pattern. Exact and semantic reuse often worked, but expected knowledge cases were frequently served through exact or semantic routes instead of the knowledge branch. From a product perspective, this can still be useful because the system reduces cost and returns the right answer. From a research perspective, it means the current experiments do not yet prove that the extracted-knowledge route is reliable. Future work should add span-supported fact extraction and stricter route isolation for knowledge-hit evaluation.

5.3 Ablation-Style Comparisons

The main ablation-style evidence so far compares three operating modes: no cache, cold cache, and warm cache. In LegalBench/CUAD, the no-cache baseline required 360 API calls and cost an estimated $1.962066. The cold-cache run reduced this to 285 API calls and $1.278803 while improving measured answer accuracy from 0.5833 to 0.6667. The warm-cache run reduced the repeated workload to 30 API calls and $0.009688 while preserving 0.6667 accuracy. This comparison suggests that cache persistence, rather than only retrieval quality, is responsible for the largest cost reduction.

The RULER cold/warm comparison isolates persistent cache reuse even more directly because the same 102-sample workload is run twice. The cold run writes 102 entries and makes 306 API calls; the warm run loads those 102 entries, obtains 102 cache hits, and makes zero API calls while preserving the same 0.8113 accuracy. This supports the core design goal: once a repeated RLM-style workload has been computed and safely scoped, future equivalent work can be served as lookup rather than regenerated.

There was also an implementation-level ablation during development. An earlier embedding design used first-token pooling for query/document embeddings. Because many queries shared the same instruction prefix, distinct queries collapsed toward nearly identical vectors, which amplified false knowledge hits. The corrected implementation uses attention-masked mean pooling, producing distinct vectors for distinct queries. This was not a full benchmark ablation, but it materially changed cache reliability and should be included as an engineering lesson: semantic caching is only as safe as the embedding and verification policy behind it.

The project still needs stronger final ablations. The next report should compare exact-only caching, vector-only

semantic caching, vector-plus-verifier caching, knowledge caching with and without source-span verification, and direct RAG without answer caching. These ablations are future work because the current project stage has focused on building the system, generating benchmark tracks, and validating the main cold/warm cache behavior.

6 Discussion and Future Work

6.1 Completed Technical Work

The project solved or made progress on several technical problems.

First, it moved from simple RLM experimentation to a reusable semantic cache architecture. Instead of treating every recursive subcall as a fresh LLM call, the system can reuse exact matches, semantic paraphrases, and extracted knowledge.

Second, it added scoped cache correctness. The system now distinguishes between exact source chunks and active document sets, reducing the risk that a cached answer from one document or benchmark sample will contaminate another.

Third, it added benchmark reproducibility infrastructure. Runs now write manifests, predictions, bridge rows, evaluation reports, cache state, and dataset signatures. This makes it possible to compare cold and warm runs and preserve historical artifacts.

Fourth, it added cost accounting. Benchmark `delta_cost_usd` and run-level totals are estimated from centralized pricing assumptions in `semantic_cache_system.py`, which keeps cost comparisons consistent.

Fifth, it identified and fixed important evaluation/caching bugs. One major bug involved embedding pooling: first-token pooling caused different queries with shared instruction prefixes to collapse toward identical vectors, amplifying false knowledge hits. The fix switched to attention-masked mean pooling, making distinct query embeddings meaningfully different. Another issue involved multi-value RULER scoring, where accuracy was not calculated using the full set of expected answers.

Sixth, it expanded the benchmark plan from one benchmark to a portfolio. RULER, NoLiMa, LegalBench/CUAD, and LongBench-v2 now each serve different purposes: retrieval stress testing, latent retrieval, real-domain route debugging, and broader long-context reasoning.

6.2 Limitations

Several limitations remain. The RULER RLM baseline is currently small. The recorded run has only 3 samples, so it cannot support a broad conclusion about RLM versus semantic-cache performance yet.

The knowledge-hit route needs more work. Synthetic and legal route diagnostics show that expected knowledge cases are often answered by exact or semantic routes instead of the knowledge branch. This may be acceptable for cost reduction, but it weakens claims specifically about extracted-knowledge reuse unless the knowledge route is improved and measured separately.

The current grounding checks are strongest for numeric facts. More robust span-based grounding is needed for document IDs, entity names, dates, titles, and exact benchmark answers. The development notes propose adding `source_doc_id`, character spans, support text, verifier status, and extraction confidence to extracted facts.

LongBench-v2 is prepared but not fully evaluated in the visible artifacts. The dataset preparation scripts and pilot generated rows are in place, but a complete benchmark run and scoring report still appear to be future work.

Some benchmark scores need careful interpretation. RULER and NoLiMa are useful retrieval stress tests, but they do not fully measure complex reasoning. CUAD has real legal text, but answer-span scoring can be brittle when compared with free-form LLM outputs. LongBench-v2 should help address this because multiple-choice labels make scoring cleaner.

6.3 Next Steps

The next steps are:

- Finish the modified LongBench-v2 benchmark runner and run the combined cache suite.

- Compare three modes on LongBench-v2: direct LLM baseline, uncached RLM baseline, and semantic cache system.
- Decide whether knowledge hits should be a required LongBench-v2 route or only reported when repeated contexts naturally support them.
- Add stronger span-based provenance for nonnumeric facts.
- Improve route telemetry so reports separate exact, semantic, knowledge, and miss behavior from answer correctness.
- Expand the RULER RLM baseline beyond the 3-sample smoke run.
- Frame the final project claim around RLM motivation while explaining that the cache infrastructure can also apply to broader autonomous-agent workflows.

7 Conclusion

This report described the semester's progress from understanding Recursive Language Models to building a semantic caching and evaluation system for recursive and autonomous LLM workflows. The central problem is that recursive decomposition can help process long inputs, but it also creates many repeated model calls. Those calls are expensive, slow, and difficult to control. The proposed solution is a scoped semantic cache that combines exact matching, embedding-based retrieval, evaluator-model verification, source/data-scope isolation, persistent cache state, and benchmark artifact logging.

The experiments completed so far show that this approach can substantially reduce repeated API usage. On RULER v2, the warm-cache run preserved the cold-cache accuracy of 0.8113 while reducing API calls from 306 to 0 and estimated API cost from \$1.212457 to \$0. On LegalBench/CUAD, the warm-cache run reduced API calls from 360 in the baseline to 30, reduced estimated cost from \$1.962066 to \$0.009688, and reduced runtime from 4,848.257 seconds to 659.463 seconds, while preserving the cache-enabled answer accuracy of 0.6667. The synthetic cache-mode suite further showed 100% answer accuracy and a 95.65% cache hit rate, but also exposed route-diagnostic issues because answer correctness and route correctness were not always aligned.

The main conclusion is that semantic caching is a promising infrastructure layer for RLM-style workloads, but the project is not finished. The current results support the cost-reduction claim for repeated workloads, while the route-level diagnostics show where the system still needs improvement. The next stage should complete the LongBench-v2 evaluation, expand the RLM baseline, add span-supported provenance for knowledge hits, and run cleaner ablations. These steps will test whether the same efficiency gains hold for broader long-context reasoning tasks, not only repeated benchmark reruns and cache-route fixtures.

8 References

[1] A. L. Zhang, T. Kraska, and O. Khattab, "Recursive Language Models," arXiv:2512.24601, 2025.

[2] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as Operating Systems," arXiv:2310.08560, 2023.

[3] Y. Bai et al., "LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding," in Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2024.

[4] C.-P. Hsieh et al., "RULER: What's the Real Context Size of Your Long-Context Language Models?" in Conference on Language Modeling (COLM), 2024.

[5] A. Modarressi, H. Deilamsalehy, F. Dernoncourt, T. Bui, R. A. Rossi, S. Yoon, and H. Schuetze, "NoLiMa: Long-Context Evaluation Beyond Literal Matching," in International Conference on Machine Learning (ICML), 2025.

[6] Y. Bai et al., "LongBench v2: Towards Deeper Understanding and Reasoning on Realistic Long-context Multitasks," in Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2025.

[7] B. Fu and D. Feng, "GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings," in Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS), 2023.

[8] W. Gill, M. Elidrisi, P. Kalapatapu, A. Ahmed, A. Anwar, and M. A. Gulzar, "MeanCache: User-Centric Semantic Caching for LLM Web Services," in Proceedings of the 39th IEEE International Parallel and Distributed Processing Symposium (IPDPS), 2025.

[9] L. G. Schroeder et al., "Adaptive Semantic Prompt Caching with VectorQ," arXiv:2502.03771, 2025.

[10] C. Jin et al., "RAGCache: Efficient Knowledge Caching for Retrieval-Augmented Generation," ACM Transactions on Computer Systems, 2024.

[11] A. K. Singh, H. Wang, L. N. S. Attaluri, T. Chiam, and W. Zhu, "Asynchronous Verified Semantic Caching for Tiered LLM Architectures," arXiv:2602.13165, 2026.

A RLM Execution Flow Diagram

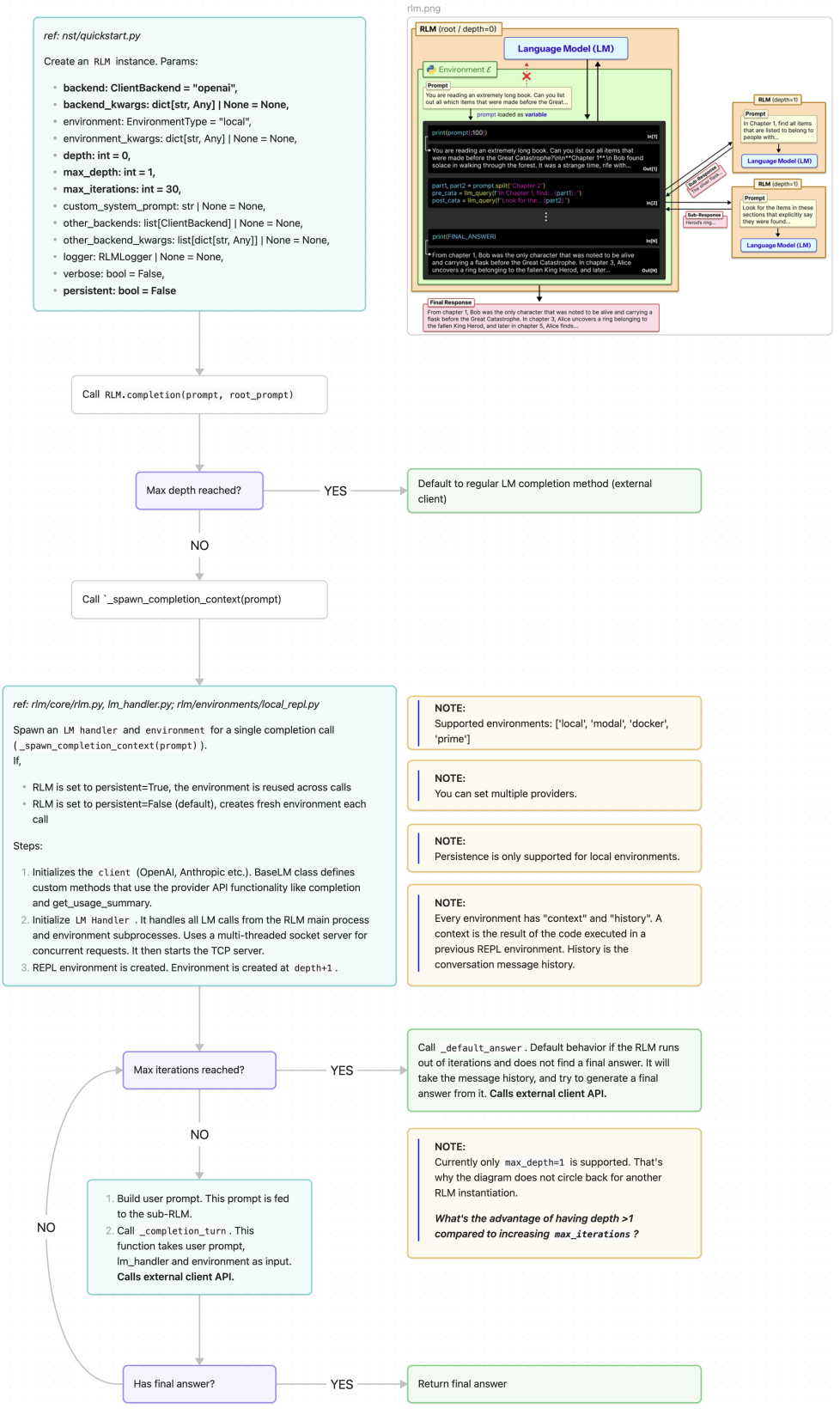

Figure 1: RLM execution flow diagram